# Nutanix NAI Chatbot

## Installation & Deployment Guide

Gradio-based AI chat interface with SQLite conversation history,
document context ingestion, and Nutanix AI Inference (NAI) integration

Document Version: 1.0
Audience: Technical / DevOps Engineers

# 1. Overview

This guide covers the complete setup and deployment of the Nutanix NAI Chatbot — a Python-based conversational AI application that integrates with Nutanix AI Inference (NAI) endpoints. The application exposes a Gradio web interface on port 7860, stores conversation history in a local SQLite database, and supports contextual document ingestion (PDF, DOCX, and plain text).

## 1.1 Architecture Summary

The application is composed of the following layers:

- Frontend: Gradio web UI (served on 0.0.0.0:7860)
- Backend API: OpenAI-compatible client pointed at a Nutanix NAI endpoint
- Persistence: SQLite database (chat_history.db) for conversation storage
- Document Ingestion: PyPDF2 and python-docx for extracting file context

## 1.2 Prerequisites

Before starting, confirm the following requirements are available on the deployment host:

| Requirement | Minimum Version | Notes |
|---|---|---|
| **Python** | 3.9+ | 3.10 or 3.11 recommended |
| **pip** | 23.0+ | Bundled with Python 3.9+ |
| **Network access to NAI endpoint** | Required | HTTP/HTTPS; self-signed certs supported via httpx verify=False |
| **NAI API Key** | Required | Obtained from the Nutanix AI console |
| **Disk space** | ~500 MB | Python env + packages + SQLite DB |

# 2. Project Setup

## 2.1 Create the Project Directory

Create a dedicated directory to contain all application files, the virtual environment, and the SQLite database.

```
mkdir nutanix-nai-chatbot
cd nutanix-nai-chatbot
```

## 2.2 Save the Application Script

Save the chatbot source code into the project directory as app.py. Ensure the file is saved with UTF-8 encoding.

```
# Place the chatbot Python source code in this file
touch app.py
# Then paste the chatbot script content into app.py using your preferred
editor
```

# 3. Python Virtual Environment

Using a virtual environment is strongly recommended. It isolates the application dependencies from your system Python installation and prevents version conflicts with other projects.

## 3.1 Create the Virtual Environment

```
# Linux / macOS
python3 -m venv venv

# Windows
python -m venv venv
```

## 3.2 Activate the Virtual Environment

The virtual environment must be activated before installing packages or running the application.

```
# Linux / macOS
source venv/bin/activate

# Windows (Command Prompt)
venv\Scripts\activate.bat

# Windows (PowerShell)
venv\Scripts\Activate.ps1
```

When active, your terminal prompt will be prefixed with (venv). All subsequent pip install and python commands will operate within this isolated environment.

### 3.3 Upgrade pip

Upgrade pip to the latest version to avoid dependency resolution issues with newer packages.

```
pip install --upgrade pip
```

## 4. Install Required Python Packages

### 4.1 Full Package List

The application depends on the following Python packages. Install all of them in a single command using the requirements file approach (recommended) or individually.

| Package | Version | Purpose |
|---|---|---|
| **`gradio`** | >=4.0 | Web UI framework for the chat interface |
| **`openai`** | >=1.0 | OpenAI-compatible API client for NAI communication |
| **`httpx`** | >=0.24 | Async HTTP client; enables TLS verification bypass for self-signed certs |
| **`python-dotenv`** | >=1.0 | Loads environment variables from .env file at runtime |
| **`PyPDF2`** | >=3.0 | Extracts text from PDF files for document context |
| **`python-docx`** | >=0.8 | Extracts text from DOCX files for document context |

**Note:** *sqlite3 is part of the Python standard library and does not require a separate installation.*

### 4.2 Install via requirements.txt (Recommended)

Create a requirements.txt file in the project root with the following contents:

```
gradio>=4.0
openai>=1.0
httpx>=0.24
python-dotenv>=1.0
PyPDF2>=3.0
python-docx>=0.8
```

Then install all packages at once:

```
pip install -r requirements.txt
```

### 4.3 Install Packages Individually

If you prefer to install packages one at a time, run each of the following commands:

```
pip install gradio
pip install openai
pip install httpx
pip install python-dotenv
```

```
pip install PyPDF2
pip install python-docx
```

### 4.4 Verify Installed Packages

Confirm that all packages are installed correctly by listing the active environment:

```
pip list
```

Cross-check that gradio, openai, httpx, python-dotenv, PyPDF2, and python-docx are present in the output.

## 5. Environment Configuration

The application reads its configuration exclusively from environment variables. The recommended approach is to define these variables in a .env file located in the project root directory. The python-dotenv library will automatically load this file when the application starts.

### 5.1 Create the .env File

Create a file named .env in the project root. Never commit this file to source control, as it contains your API key.

```
touch .env
```

### 5.2 Required Environment Variables

Populate the .env file with the following key-value pairs, replacing the example values with your actual Nutanix NAI credentials:

```
NAI_BASE_URL=https://<your-nai-endpoint>/api/v1
NAI_API_KEY=<your-nai-api-key>
NAI_MODEL=<model-name-deployed-on-nai>
```

| Variable | Example Value | Description |
|---|---|---|
| **NAI_BASE_URL** | `https://nai.corp.local/api/v1` | Base URL of the Nutanix NAI OpenAI-compatible endpoint. Trailing slashes are stripped automatically. |
| **NAI_API_KEY** | `nai-xxxxxxxxxxxxxxxx` | API key issued by the Nutanix AI console for authentication. |
| **NAI_MODEL** | `llama3-8b-instruct` | The model identifier as registered in the NAI deployment. |

> **Warning:** NAI_BASE_URL must point to the OpenAI-compatible /v1 path of your NAI endpoint. Omitting this path is the most common cause of 404 errors on first run.

### 5.3 TLS Certificate Handling

The application is pre-configured to bypass TLS certificate verification via httpx.Client(verify=False). This is intentional for environments where NAI is deployed with a self-signed or internal CA certificate. If your NAI endpoint uses a publicly trusted certificate, you may optionally change this to verify=True in the source code for enhanced security.

### 5.4 .gitignore Recommendation

If using version control, add the following entries to your .gitignore file to prevent accidental exposure of secrets:

```
.env
chat_history.db
venv/
__pycache__/
*.pyc
```

## 6. Running the Application

### 6.1 Start the Server

With the virtual environment active and the .env file in place, start the application using:

```
python app.py
```

On successful startup, you will see output similar to the following in your terminal:

```
Running on local URL:  http://0.0.0.0:7860
Running on public URL: https://xxxxxxxx.gradio.live
```

### 6.2 Accessing the Interface

Open a browser and navigate to http://localhost:7860 (or the host IP address if accessing remotely). The Gradio chat interface will be displayed.

### 6.3 Application File Locations

The following files are created automatically at runtime:

- chat_history.db — SQLite database file created in the working directory on first run
- No additional directories are required; Gradio serves the UI from memory

## 7. Verifying Functionality

### 7.1 Basic Chat Test

1. Navigate to http://localhost:7860 in your browser.
2. Type a message in the Message box and press Enter.

3. Verify that the assistant returns a response from the NAI endpoint.
4. Observe that the conversation appears in the History dropdown (top-left panel).

### 7.2 Document Context Test

5. Click the Context file upload area in the left panel.
6. Upload a PDF, DOCX, or plain text file.
7. Ask a question related to the document content.
8. Verify that the assistant references information extracted from the file.

### 7.3 Persistence Test

9. Start a new conversation using the New Chat button.
10. Send a few messages.
11. Stop the application (Ctrl+C) and restart it with python app.py.
12. Open the History dropdown and confirm previous conversations have been restored.

## 8. Troubleshooting

| Symptom | Likely Cause | Resolution |
|---|---|---|
| ModuleNotFoundError on startup | Virtual env not activated or package missing | Activate venv and re-run pip install -r requirements.txt |
| 401 Unauthorized from NAI | Incorrect or expired API key | Regenerate NAI_API_KEY and update .env |
| 404 Not Found on API call | Incorrect NAI_BASE_URL path | Ensure URL ends with /api/v1 or the correct versioned path |
| SSL certificate verification error | Unexpected TLS verification enabled | Confirm httpx.Client(verify=False) is set in the client init |
| Port 7860 already in use | Another service is bound to port 7860 | Change server_port in demo.launch() or stop the conflicting process |
| client not defined error | OpenAI client line is commented out in source | Uncomment the client = OpenAI(...) line in the Config section |

## 9. Security Considerations

Review the following security notes before deploying in a production or shared environment:

- TLS Verification: The current configuration disables certificate verification (verify=False). This is suitable for internal lab environments but should be replaced with a proper CA bundle in production.
- API Key Exposure: The .env file contains a live API key. Ensure file permissions restrict read access (chmod 600 .env on Linux/macOS).

- Network Binding: The server binds to 0.0.0.0:7860, making it accessible on all network interfaces. Restrict this using a firewall or reverse proxy (e.g., nginx) in production.
- Database Security: The SQLite database (chat_history.db) stores all conversation history in plaintext. For sensitive deployments, consider encrypting the database or migrating to a secured database backend.
- Gradio Public Link: If Gradio generates a public sharing URL, it will be printed to the console. Disable this in production by setting share=False in demo.launch().

## 10. Quick-Start Checklist

Use the following checklist to validate a clean installation:

| | Step |
|---|---|
| ☐ | Create project directory and save app.py |
| ☐ | Create and activate Python virtual environment (python3 -m venv venv) |
| ☐ | Upgrade pip (pip install --upgrade pip) |
| ☐ | Install all dependencies (pip install -r requirements.txt) |
| ☐ | Create .env file with NAI_BASE_URL, NAI_API_KEY, and NAI_MODEL |
| ☐ | Confirm NAI endpoint is reachable from the host |
| ☐ | Uncomment the client = OpenAI(...) line in app.py |
| ☐ | Run the application (python app.py) |
| ☐ | Verify chat interface loads at http://localhost:7860 |
| ☐ | Send a test message and confirm NAI responds |
| ☐ | Test document upload and confirm context is applied |
| ☐ | Verify conversation history persists after restart |